IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

FPGA accelerator for LLM

Report of week 3

Work of :

Gauvrit Jean-Luc

As intern student

With the supervision of :

Kim Tae-Wan

June 2025

Contents

List of Figures

1 Introduction

1.1 Presentation of the Weekly Report

This weekly report presents the progress of my work on designing and optimizing a hardware accelerator for Large Language Models (LLMs) using Field-Programmable Gate Arrays (FPGAs). Its objective is to document technical advancements, highlight open design questions, and ensure alignment with the research direction defined in collaboration with Professor Tae-Wan Kim. Each section outlines key investigations, experimental results, and technical challenges encountered during the week.

1.2 Presentation of the Subject

The goal of this project is to do inference with a LLM, specifically a variant of the DeepSeek-R1 family, on a FPGA platform. DeepSeek has recently demonstrated excellent inference performance across various benchmarks, offering competitive accuracy while being more lightweight and efficient than many other open-source LLMs of similar size. These characteristics make it a strong candidate for hardware-level optimization.

LLMs typically require considerable computational power and memory bandwidth, which limits their deployment in edge or low-power environments. While GPUs provide strong performance, their energy consumption is often prohibitive for embedded or mobile use cases.

This project explores the use of FPGAs to design a custom, energy-efficient accelerator optimized for DeepSeek inference. The flexibility of FPGAs allows for fine-tuned architectural choices targeting both compute and memory efficiency. Our focus is on reducing power consumption while maintaining acceptable latency (e.g., 100–200 ms per token), with attention given to constraints such as voltage levels, memory interfaces, and model size.

1.3 The model

The goal of this project is to perform inference with a Large Language Model, specifically a variant of the DeepSeek-R1 family, on an FPGA platform. DeepSeek has recently demonstrated excellent inference performance across various benchmarks, offering competitive accuracy while being more lightweight and efficient than many other open-source LLMs of similar size. These characteristics make it a strong candidate for hardware-level optimization.

Figure 1: Logo de DeepSeek

Ultimately, the objective is to evaluate whether FPGA-based acceleration can provide a sustainable and scalable solution for running LLMs like DeepSeek in compact or embedded systems.

1.4 Goals

Our final goal is to develop an Application-Specific Integrated Circuit (ASIC) that is energy-efficient and capable of accelerating LLMs in low-power environments. The development of a

Proof of Concept (PoC) using FPGAs is a crucial step towards achieving this goal. The PoC will demonstrate the feasibility and potential benefits of using FPGAs for LLM acceleration, paving the way for future ASIC development.

1.5 FPGA

Field-Programmable Gate Arrays (FPGAs) are reconfigurable integrated circuits that can be programmed to implement custom digital logic. Their main advantage lies in flexibility : unlike fixed-function hardware, FPGAs allow designers to create architectures tailored to specific applications, such as neural network inference. This adaptability makes them well suited for achieving an efficient balance between performance, power consumption, and resource usage.

2 Softmax

2.1 Optimization

The `softmax` function plays a central role in attention mechanisms by transforming input scores into a probability distribution:

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}} \tag{1}$$

To improve hardware efficiency, particularly on FPGA platforms, we explore the following three optimization strategies:

- **Remove the division step** to avoid expensive normalization operations.
- **Approximate the exponential function** e^x using low-order polynomial expansions (e.g., Taylor series).
- **Token-level decision logic**, where the softmax weight computation is merged with the selection process [2].

2.2 Simulations

To evaluate the computational efficiency of various normalization algorithms, we implemented and benchmarked several alternatives to the standard `Softmax`, including *FlashNorm* [2], a Taylor-based exponential approximation, Softmax with translation (log-sum-exp trick), and the `LogSumExp` function.

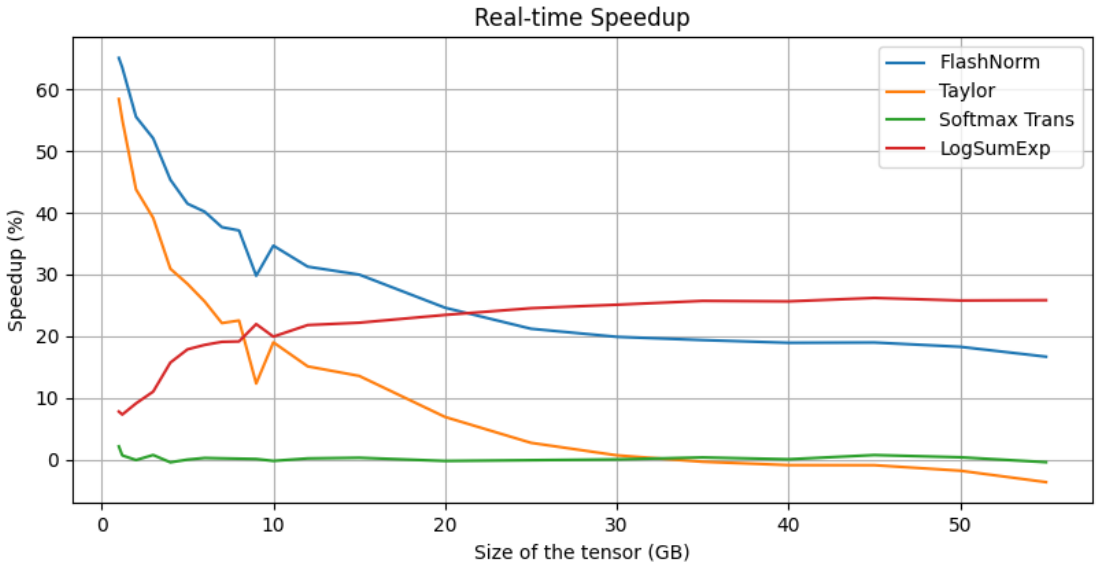


Figure 2: Simulation of Softmax algorithms running on CPU

As shown in Figure 2, some inconsistencies are noticeable in the CPU simulation results. For instance, Softmax with translation—which is theoretically more stable and efficient—did not yield a measurable performance improvement. These discrepancies may be due to memory management overhead or compiler-level optimizations that mask the benefits of this technique.

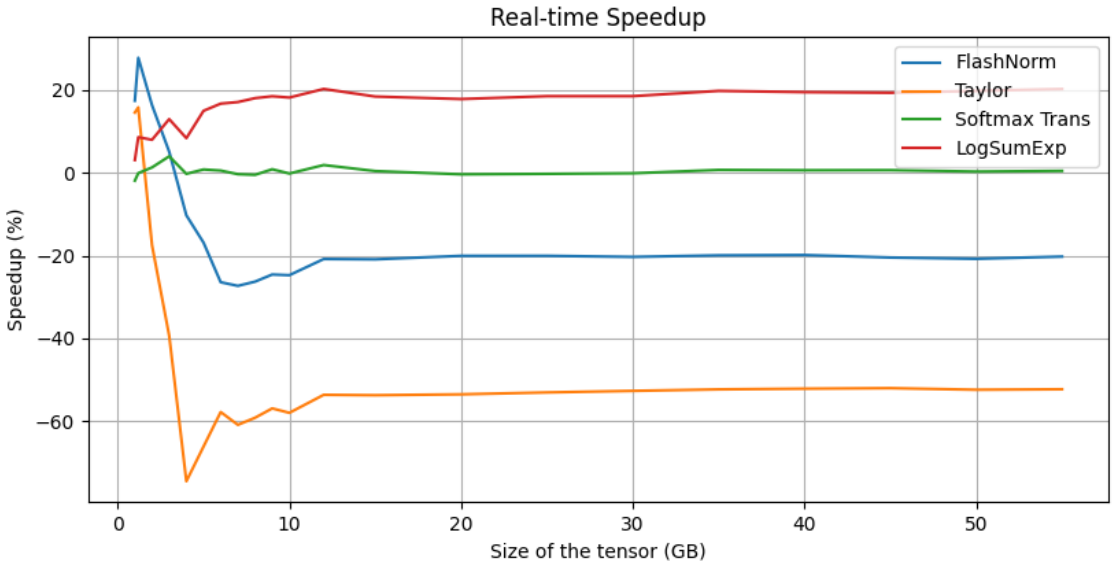


Figure 3: Simulation of Softmax algorithms running on GPU (via PyTorch)

In contrast, Figure 3 shows preliminary GPU-based results. However, these results are currently not reliable, as the implementations of the algorithms require further corrections and optimization. **This part is still under development and will be updated in future iterations of the report.**

3 Hardware Design

3.1 Peer-to-Peer Communication between GPU and FPGA

3.1.1 GPUDirect

As described [3], GPUDirect RDMA enables direct peer-to-peer data transfer between devices on the PCIe bus.

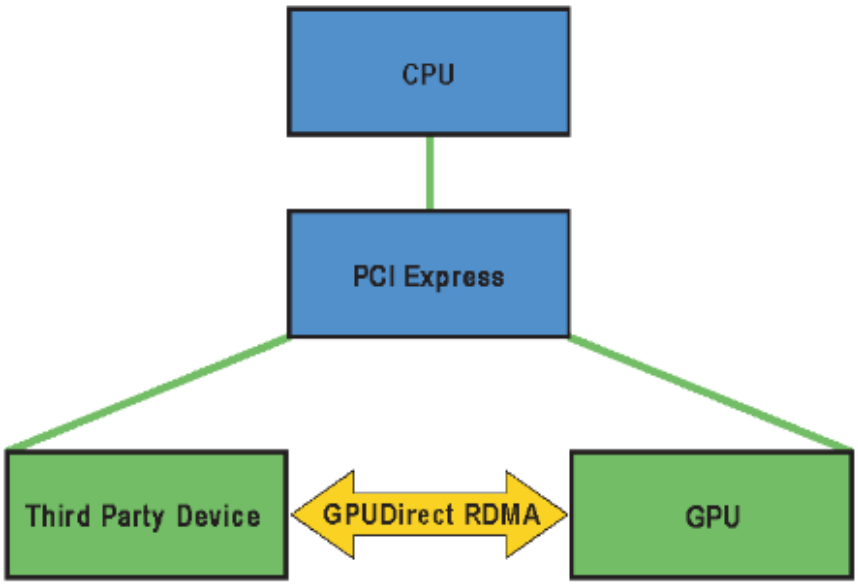


Figure 4: GPUDirect RDMA within the Linux Device Driver Model

Figure 4 presents a hardware architecture that utilizes **GPU Direct RDMA**. In this setup, a third-party device, such as an FPGA, a specialized accelerator, or a data acquisition card is connected to the same PCIe bus as the GPU.

By leveraging **GPU Direct RDMA**, the GPU can directly access the third-party device's memory **without passing data through the CPU or the system's main memory**.

This architecture offers several key benefits:

- **Reduced latency**, as data can be exchanged directly over the PCIe bus.
- **Increased bandwidth**, since the system's RAM is bypassed.
- **Lower CPU overhead**, allowing the CPU to focus on other workloads.

Overall, **GPU Direct RDMA is highly beneficial for real-time and bandwidth-intensive applications, such as machine learning inference, signal processing, and video streaming.**

3.1.2 Streaming video case

Figure 5 illustrates the peer-to-peer (P2P) architecture introduced earlier with GPUDirect RDMA.

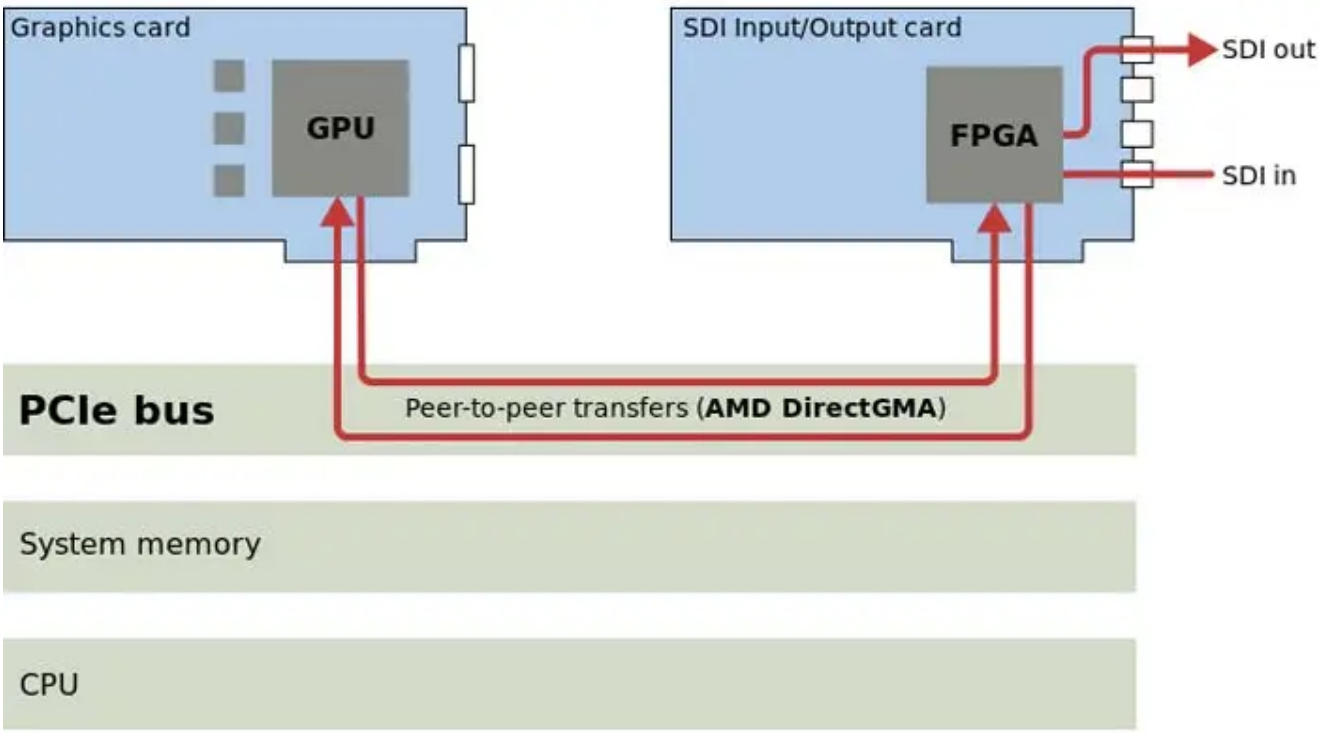


Figure 5: Peer-to-Peer Communication between GPU and FPGA using PCIe for video streaming

This setup allows data to be exchanged directly between the FPGA and GPU, avoiding intermediate copies through system memory or the CPU.

Such an architecture is especially advantageous for LLM inference workloads:

- The GPU efficiently executes highly parallel operations, including attention matrix multiplications and dense-layer computations.
- The FPGA accelerates lightweight, deterministic tasks, such as normalization and softmax approximations.
- The PCIe interface enables low-latency, high-bandwidth data transfer.

3.2 Hardware Implementation Overview

Figure 6 depicts the hardware architecture of the design. The DDR4 is connected through an AXI interconnect, allowing direct access to the MPSoC processing system. Initially, all clock domains were synchronized to the MPSoC's primary clock; however, this will be revised to use a dedicated clock wizard that supplies optimized clock signals for the DDR4 and PCIe subsystems.

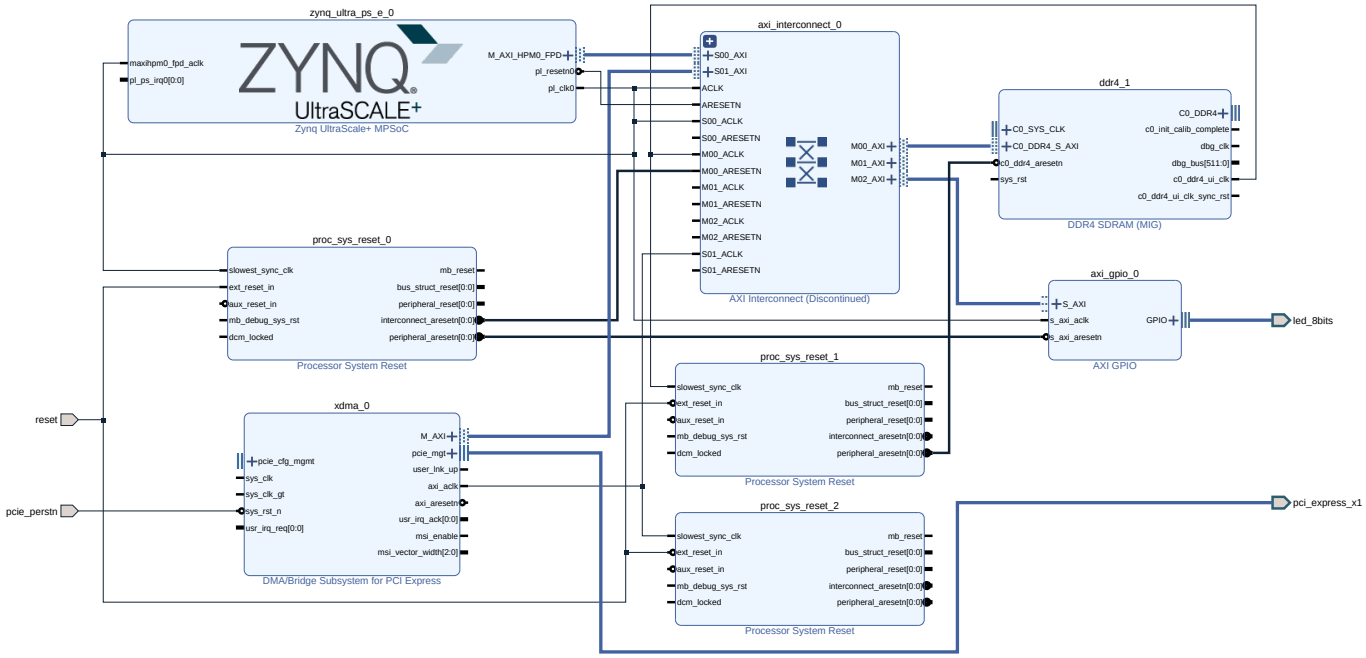


Figure 6: Hardware architecture

Additionally, the PCIe interface must be reconfigured to properly support direct communication between the MPSoC and the memory controller. Finally, a set of LEDs has been incorporated into the design as visual indicators of system status, allowing for easier debugging and real-time monitoring of the FPGA's operation during inference workloads.

3.2.1 Third party with CUDA

Figure 7 depicts a hardware architecture that enables direct data exchange between a third-party device, such as an FPGA, and the GPU without relying on intermediate CPU or host memory transfers.

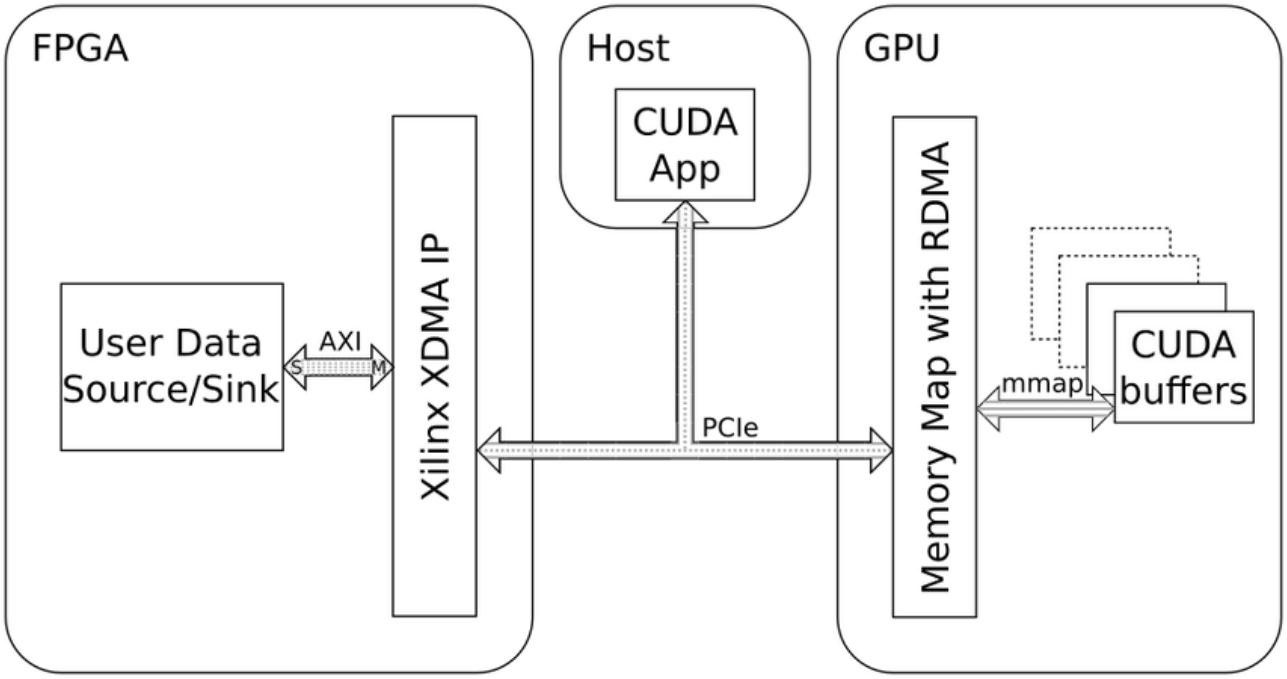


Figure 7: Direct Communication between FPGA and GPU using CUDA and PCIe

In this setup:

- The **FPGA**, equipped with the `XDMA` core, handles data ingress and egress over the PCIe interface.
- The **host application**, implemented in `CUDA`, orchestrates data transfers and manages GPU buffer allocation.
- **GPUDirect RDMA** maps GPU buffers directly into the FPGA's addressable memory space, allowing data to flow between devices without CPU staging.

This direct data path reduces CPU intervention and memory copies, resulting in lower latency and higher throughput. The architecture is well-suited for real-time signal processing and deep learning inference workloads that require both the high computational power of a GPU and the deterministic, low-power processing of an FPGA.

4 Conclusion

This report outlines the progress achieved during the third week of the FPGA accelerator project for Large Language Models (LLMs). We investigated optimization techniques for the `Softmax` function, performed simulation-based analyses on CPU and GPU, and proposed a hardware architecture that enables efficient FPGA–GPU peer-to-peer communication via PCIe.

The proposed architecture exploits the strengths of both devices: the GPU efficiently handles compute-intensive matrix multiplications, while the FPGA provides low-latency acceleration for operations such as normalization and customized post-processing. Direct peer-to-peer transfers minimize CPU overhead and reduce data movement latency, making this setup highly suitable for real-time LLM inference in embedded or energy-constrained environments.

Future work will focus on:

- Completing the hardware design by properly configuring the AXI interconnect and synchronizing the clock domains for DDR4 and PCIe.
- Implementing the proposed Softmax optimizations in hardware to remove the costly division operations and approximate the exponential efficiently.
- Validating the design on the ZCU106 evaluation board and measuring end-to-end performance and energy consumption.

This proof of concept aims to establish a practical path toward more energy-efficient LLM inference on FPGAs, paving the way for a future ASIC implementation with further power and cost optimizations.

5 Annex

Cloud FPGA architectures leverage the flexibility and computational power of reconfigurable hardware, combined with the speed of ASICs for high-speed I/O, allowing acceleration of compute, storage, and networking workloads across data center environments. However, deployment must be carefully planned to balance cost, power, cooling, compatibility, and scalability. Different FPGA architectures present distinct advantages depending on the system requirements — from co-processor setups and distributed clusters to highly disaggregated or network-centric designs, all aimed at delivering optimal performance for diverse cloud workloads.[1]

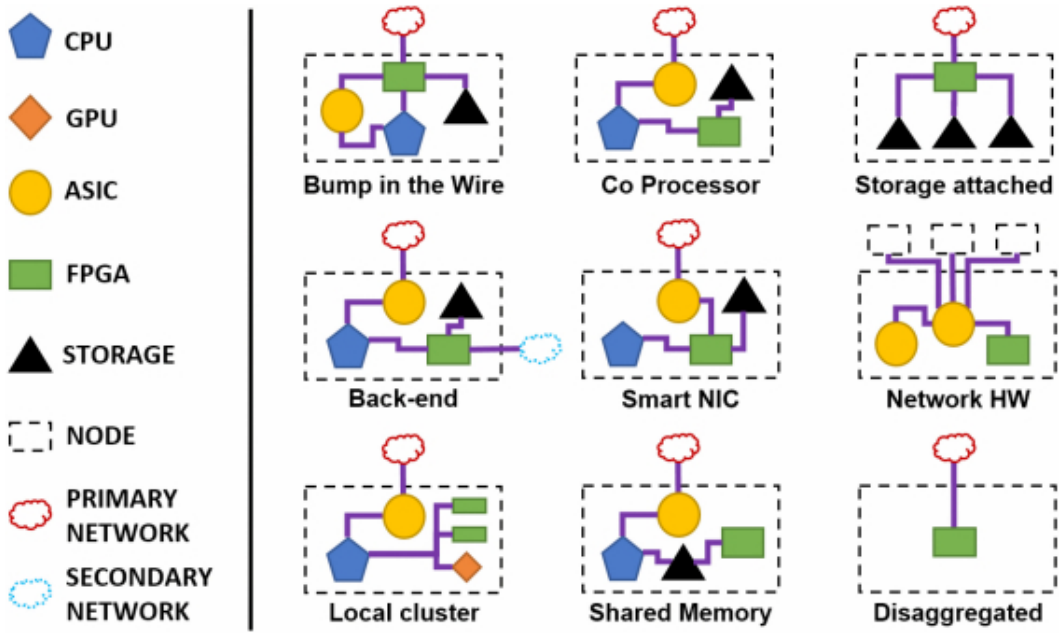
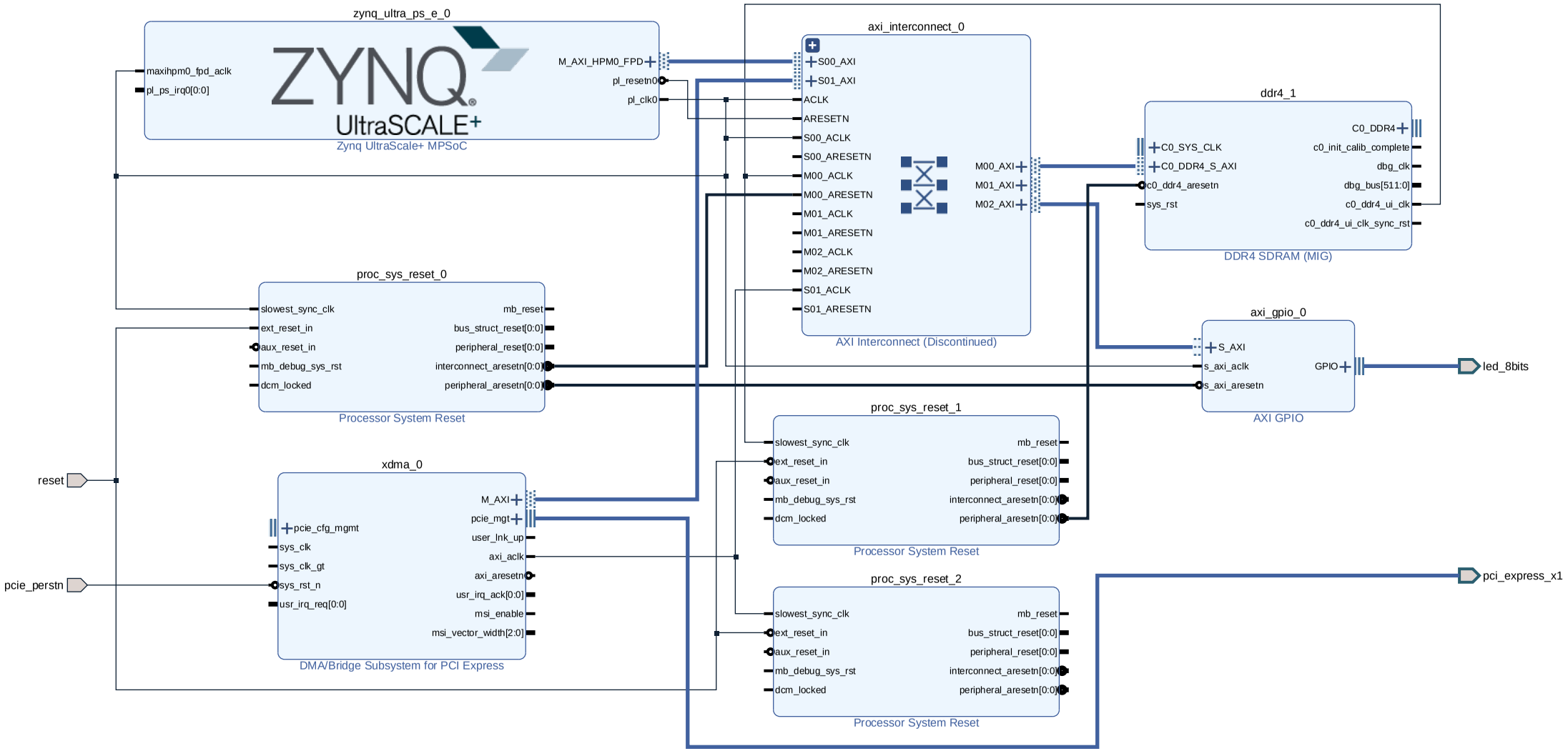


Figure 8: Future architectures of Cloud

A taxonomy of cloud FPGA architectures is defined by five key factors : the type of FPGA boards used, their placement in the system, their network connectivity, their intra-node connectivity, and the intended use cases. These criteria shape the design choices, from off-the-shelf to fully customized solutions, distributed or centralized placement, and varying levels of interconnect complexity between FPGAs, CPUs, GPUs, or other devices. The resulting architectures reflect trade-offs between performance, cost, upgradability, and scalability, and help inform future trends in FPGA-enabled cloud infrastructure.


zynq_ultra_ps_e_0
maxihpm0_fpd_aclk
pl_ps_irq0[0:0]
M_AXI_HPM0_FPD
pl_resetn0
pl_clk0
Zynq UltraScale+ MPSoC
axi_interconnect_0
S00_AXI
S01_AXI
ACLK
ARESETN
S00_ACLK
S00_ARESETN
M00_ACLK
M00_ARESETN
M01_ACLK
M01_ARESETN
M02_ACLK
M02_ARESETN
S01_ACLK
S01_ARESETN
M00_AXI
M01_AXI
M02_AXI
AXI Interconnect (Discontinued)
ddr4_1
C0_SYS_CLK
C0_DDR4_S_AXI
c0_ddr4_aresetn
sys_rst
C0_DDR4
c0_init_calib_complete
dbg_clk
dbg_bus[511:0]
c0_ddr4_ui_clk
c0_ddr4_ui_clk_sync_rst
DDR4 SDRAM (MIG)
proc_sys_reset_0
slowest_sync_clk
ext_reset_in
aux_reset_in
mb_debug_sys_rst
dcm_locked
mb_reset
bus_struct_reset[0:0]
peripheral_reset[0:0]
interconnect_aresetn[0:0]
peripheral_aresetn[0:0]
Processor System Reset
axi_gpio_0
S_AXI
s_axi_aclk
s_axi_aresetn
GPIO
AXI GPIO
led_8bits
reset
xdma_0
pcie_cfg_mgmt
sys_clk
sys_clk_gt
sys_rst_n
usr_irq_req[0:0]
M_AXI
pcie_mgt
user_lnk_up
axi_aclk
axi_aresetn
usr_irq_ack[0:0]
msi_enable
msi_vector_width[2:0]
DMA/Bridge Subsystem for PCI Express
pcie_perstn
proc_sys_reset_1
Processor System Reset
proc_sys_reset_2
Processor System Reset
pci_express_x1

References

[1] Christophe Bobda, Joel Mandebi Mbongue, paul chow, Mohammed Ewais, Naif Tarafdar, Juan Camilo Vega, Ken Eguro, Dirk Koch, Suranga Handagala, Miriam Leeser, Martin Herbordt, Hafsah Shahzad, Peter Hofste, Burkhard Ringlein, Jakub Szefer, Ahmed Sanaullah, and Russell Tessier. The future of fpga acceleration in datacenters and the cloud. *ACM Transactions on Reconfigurable Technology and Systems*, 15:1–42, 02 2022.

[2] Nils Graef, Andrew Wasielewski, and Matthew Clapp. Flashnorm: fast normalization for llms, 2025.

[3] NVIDIA Corporation. *GPUDirect RDMA*. Santa Clara, CA, May 2025. Accessed on June 2025.